

Name: Maria Remedios

Apply Position: Senior QA Engineer

Date: January 29, 2026

=====

Take-home Assignment

Question 1

Part 1 - The booking flow is integral to Ezra's business operation. Please go through the first three steps of the booking process including payment and **devise 15 test cases** throughout the entire process you think are the most important. When submitting the assignment, please return the test cases from the most important to the least important.

Given:

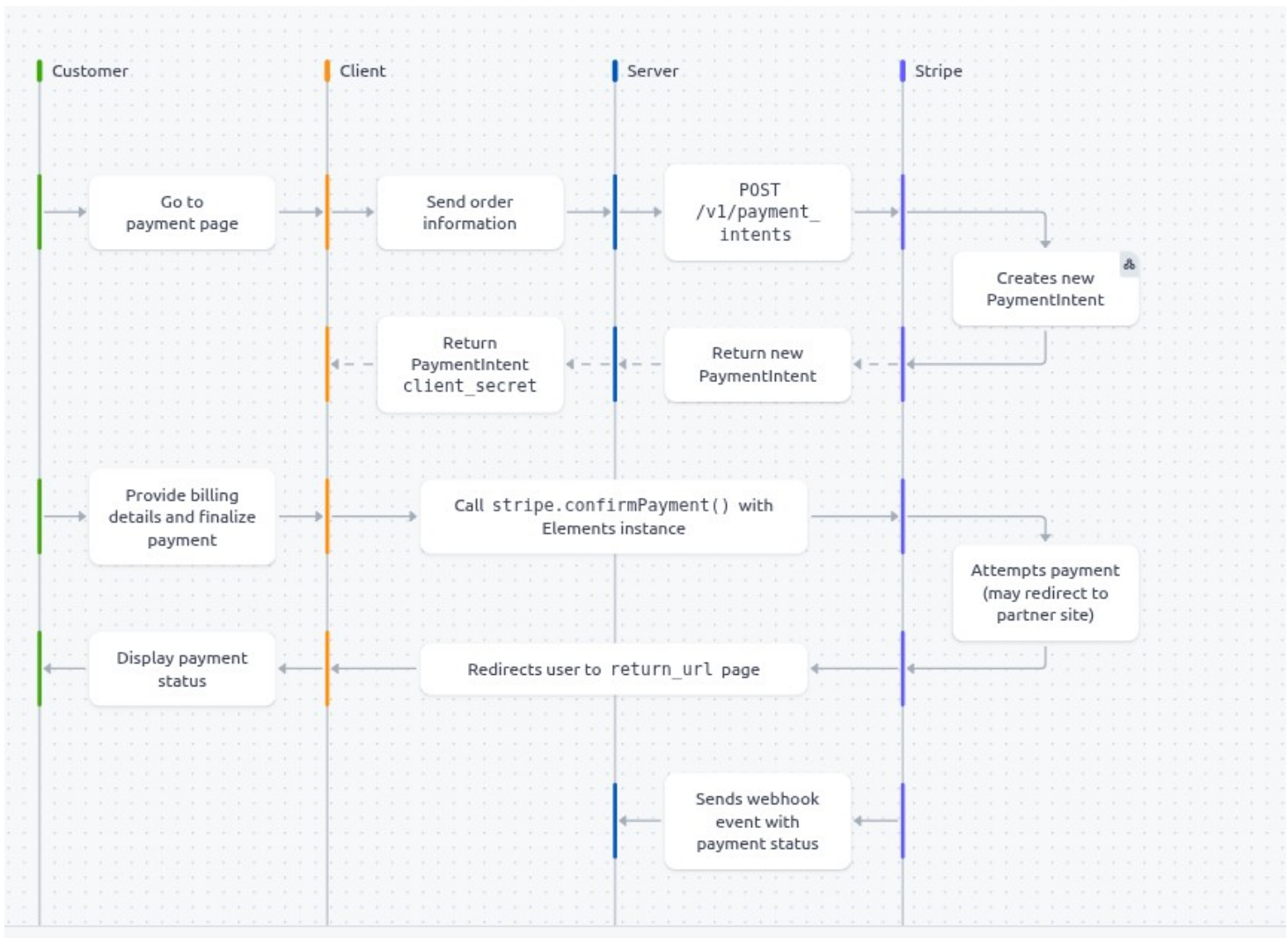
The following are the Booking portal URLs to Ezra's business operation system:

1. Member Facing Portal URL
2. User Facing Portal URL
3. Strip Test Data

High-level Ezra Booking System process flow:

A **PaymentIntent** collects a payment from a customer. System creates exactly one PaymentIntent for each order or customer session.

A **PaymentIntent** changes status during the payment process. It works with *stripe.js* to handle things like customer authentication and will only have one successful charge.



Below are the most important **15 test cases** throughout the whole booking process:

Test Case ID:	Priority	Test Case Description	Preconditions	Testing Steps	Test Data	Expected Result
TC-01	1	Verify end-to-end booking and payment charging flow is working	1. A valid user has been created and successfully login to the system 2. User is on "Booking" page	1. Select booking service, date, time 2. Review booking 3. Enter valid Stripe card 4. Complete payment 5. Confirm the booking and payment method 6. System returns valid booking confirmation no.	Valid Stripe test card	1. No error. • The charge appears as Succeeded in the Dashboard under Payment page • Stripe captures the charge. 2. Booking is confirmed 3. Customer received booking email Confirmation
TC-02	2	Verify booking fail if payment fails	User is on "Payment" page	1. Complete booking details 2. Enter invalid Stripe card information or Invalid payment method 3. Submit payment	Stripe declined test card	1. Payment fails and booking fail 2. Customer receives booking fail email notification
TC-03	3	Verify system behavior when payment authorization success but booking confirmation fails	User is on "Payment" page	1. Complete booking 2. Complete payment 3. Simulate booking failure	Stripe valid test card	User receives email notification for payment but booking fail
TC-04	4	Verify system is able to prevent duplicate payment	User is on "Payment" page	1. User accidentally clicked "Pay" button multiple times	Stripe valid test card	Only one payment and one booking are created
TC-05	5	Verify mandatory booking fields are required	User is on "Booking Details" page	1. Leave all mandatory fields empty 2. Press "Continue" button	Empty required fields	Validation error displayed and user cannot proceed
TC-06	6	Verify unavailable date or time cannot be booked	Booking calendar is available	1. Select unavailable date/time 2. Click "Continue" button	Unavailable slot	System shows slot unavailable message
TC-07	7	Verify Booking Review page displays correct and detail information	Booking details entered	1. In the Booking Review page, enter all required information 2. Click "Continue" button	Selected booking details	Correct booking summary is displayed
TC-08	8	Verify booking price calculation is correct	Pricing rules configured	1. Select multiple booking trips 2. Review booking information and price	Multiple quantities	Total price is calculated correctly
TC-09	9	Verify price consistency between "Booking Review" and "Payment" pages	"Booking Review" page displayed	1. Note Booking currency and price 2. Proceed to "Payment" page	Stripe test card	Stripe charge matches review price

Test Case ID:	Priority	Test Case Description	Preconditions	Testing Steps	Test Data	Expected Result
TC-10	10	Verify user can edit booking details	User is on "Review" page	1. Go "Back" button 2. Modify booking information 3. Click "Continue" to "Booking Review" page	Updated booking details	Updated details and price are displayed
TC-11	11	Verify system rejects invalid Stripe card information	User is on "Payment" page	1. Enter invalid Stripe card information 2. Submit payment	Stripe invalid card	Payment fails with clear error message
TC-12	12	Verify system allows user to enter correct payment information after failure	Initial payment failed	1. Fail payment 2. Retry with valid Stripe card information	Stripe declined + valid card	Payment succeeds and booking is confirmed
TC-13	13	Verify Stripe 3D Secure authentication flow	3D Secure enabled	1. Use 3D Secure card 2. Complete authentication	Stripe 3D Secure test card	Payment succeeds after authentication
TC-14	14	Verify refreshing Booking Confirmation page does not duplicate booking	Booking completed	1. Refresh confirmation page	N/A	No duplicate booking is created
TC-15	15	Verify after pressing the back button during payment does not confirm booking	User is on "Payment" page	1. Click browser back	N/A	No booking or payment is created

Part 2 - *For the top 3 test cases from part 1, please provide a description explaining why they are indicated as your most important.*

TC-01 - Verify end-to-end booking and payment charging flow is working

This test is important because it validates the core business end-to-end flow of the booking system. If a user cannot successfully complete a booking and payment, the business cannot generate revenue. It confirms that all three booking steps—booking details, booking review, and payment, and a successful booking is created only after a successful payment.

TC-02 - Verify booking fail if payment fails

This test is important because it ensures the system correctly handles payment failure scenarios. Confirming a booking without successful payment can lead to financial loss and operational issues. This test protects the business from unpaid bookings and ensures proper validation between payment status and booking confirmation.

TC-03 - Verify system behavior when payment succeeds but booking confirmation fails

This test is important because it addresses a critical edge case where payment is successful but the booking is not confirmed due to any system issue. This scenario can result in users being charged without receiving a booking, leading to customer complaints, refunds, and loss of trust. Ensuring the system handles this scenario correctly is critical for both customers, financial integrity and all related parties.

Question 2

Part 1 -- *Being privacy focused is integral to our culture and business model. Please devise an integration test case that prevents members from accessing other's medical data.*

Hint: *Begin Medical Questionnaire.*

Integration Test Case: Prevent Access to Other Members' Medical Data

Test Case ID:	Priority	Test Case Description	Preconditions	Testing Steps	Test Data	Expected Result
INT-1	1	Verify a member cannot access another member's medical questionnaire data when beginning a medical questionnaire	1. Member A and Member B both exist in the system 2. Member B has an existing medical questionnaire saved in the system 3. Member A is logged in successfully.	1. Enter invalid Stripe card information 2. Submit payment 1. Log in to the Member Facing Portal as Member A. 2. Navigate to Begin Medical Questionnaire. 3. Intercept or manipulate the request (e.g., questionnaire ID or member ID) to reference Member B's medical questionnaire. 4. Submit the request to retrieve or continue the medical questionnaire. 5. Observe the system response.	Member A ID (logged-in user) Member B ID (different member) Existing medical questionnaire associated with Member B	The system denies access to the medical questionnaire. No medical data belonging to Member B is returned. An authorization error (e.g., 403 Forbidden or generic access denied message) is returned. The access attempt is logged for audit and security monitoring.

Part 2 -- Please devise HTTP requests from Part 1 to implement your test case. Submitting written HTTP requisitions is fine, you do not need to submit a postman project.

HTTP Requests to Prevent Access to Other Members' Medical Data

Test Description:

- Member A is a logged-in user
- Member B owns a different medical questionnaire
- This test verifies that Member A cannot access Member B's medical data, even by manipulating requests

Request 1: Begin Medical Questionnaire (**Happy** Path Access)

Test Description: Verify normal, authorized HTTP Request behavior

Sample Request:

```
POST /api/medical-questionnaire/begin
Host: api.ezra.com
Authorization: Bearer <Member_A_Access_Token>
Content-Type: application/json
{
  "memberId": "test-123"
}
```

Expected Response

```
HTTP/1.1 200 OK
```

Expected Result:

1. A new medical questionnaire session is created for Member A
2. Member A is not able to access other member's data

Request 2: Verify unauthorized access to Another Member's Questionnaire

Test Description:

This request simulates an unintended attempt to access other member's data by changing the member ID.

Sample Request:

```
POST /api/medical-questionnaire/begin
Host: api.ezra.com
Authorization: Bearer <Member_A_Access-Token>
Content-Type: application/json
{
    "memberId": "test-456"
}
```

Expected Response:

HTTP/1.1 403 Forbidden

(or "401 Unauthorized" depends on the implementation)

Expected Result:

- System returns Authorization error and denied the request
- System does not return Member B medical questionnaire data
- System enforces member-level data isolation
- This attempt is logged for security auditing

Part 3 -- At Ezra, we have over 100 endpoints that transfer sensitive data.

What is your thought process around managing the security quality of these endpoints?

Answer:

With over 100 endpoints transferring sensitive data, security quality must be managed logically and systematically.

My approach would be layered and risk-based, focusing on preventing data leakage even when clients behave unexpectedly or maliciously.

Key principles:

1. Centralized Authorization Enforcement

- Authorization must be enforced on the server-side using the authenticated identity (token/session).
- Prevent IDOR (Insecure Direct Object Reference) vulnerabilities by scoping data access to the authenticated user.

2. Reusable Security Test Patterns

- Define common negative and edge testing scenarios like : ID tampering, missing token, expired token, etc
- Reuse these patterns across all sensitive endpoints.

3. Automation at the API Layer

- UI tests alone are insufficient.
- API-level integration tests validate real security enforcement.

4. Risk-Based Prioritization

- Prioritize endpoints that:
 - Handle medical data
 - Handle payment or identity data
 - Allow data retrieval or modification

What are the tradeoffs and potential risks of your solution?

Trade-offs

Trade-off	Explanation
Coverage vs Speed	Full security testing of all endpoints is time-consuming
Automation complexity	Auth setup and token handling adds complexity
False sense of safety	Automated tests don't replace security reviews or penetration testing

Potential Risks

- Missed edge cases (e.g. new endpoints not covered by shared rules)
- Over-reliance on automation without manual security reviews
- Token misconfiguration leading to incomplete test coverage

Mitigation

- Central test utilities for auth and identity
- Use LLM technique to find high-risk testing areas in order to reduce and prevent the issues found in production
- Continue learning
- Security checks added to CI/CD
- Regular endpoint inventory reviews

Automation Using Playwright

Why I chose these test cases to automate ?

I automated the following 2–3 high-value, stable, and business-critical flows with Playwright:

Test Case no	Test Description	Reason to choose
TC-01	Verify end-to-end booking and payment charging flow is working	• Core revenue path • High regression risk
TC-02	Verify booking fail if payment fails	• Prevents financial loss • High business impact
TC-03	Verify system behavior when payment authorization success but booking confirmation fails	• Prevents over/under-charging • Stable UI assertions

Reason to the above tests for Automation:

1. They are the system main flows and are executed frequently
2. Email notifications would be sent when broken
3. Provide strong confidence when passing

Automation Assumptions

1. Stable test environment
2. Stripe test mode enabled
3. Assumed the test data is able to give the same testing result
4. Assumed the testing environment is stable without CAPTCHA and not require multi-factor authentication (MFA)

Automation Trade-offs

Trade-off	Explanation
UI automation speed	Slower than API tests
Maintenance cost	UI changes can break tests
Payment flakiness	Requires mocked or test Stripe cards

Scalability Decisions

- Page Object Model (POM)
- Reusable helpers for auth & payment
- Clear separation of test logic and UI selectors

Below is the GitHub Repo URL :

<https://github.com/horselopes-cpu/ezra-automation/blob/main>

The above GitHub Repo included the following:

- All related tests and documentation
- A short README.md with setup steps and project explanation notes.
- Comments in the scripts explaining assumptions, scalability, and what needs to be implement in the future.
- Automation scripts and structure with a scalable Page Object Model
- This project demonstrates the testing architecture, coding, and design decisions required for production-level.